FinPulse — Project Report

SoFI AlgoLabs Core Induction — Assignment 1 | Armaann Bhansali | GitHub: github.com/armaannbhansali/finpulse

Overview

FinPulse is a stock market monitoring platform that tracks 20 NSE-listed Indian companies. It fetches live market data, stores it in a database, and serves it through both a REST API and an interactive dashboard. The API and dashboard are deployed as two independent services: **Dashboard** — finpulse-27phmb7xrifs4vg9xfnrge.streamlit.app | **API** — finpulse-api-yjel.onrender.com (docs at /docs).

Architecture

The project is split into three independent pieces. **algo1.py** fetches data for the 20 tracked tickers via yfinance and writes it into a local SQLite database (finpulse.db), using an INSERT ... ON CONFLICT DO UPDATE pattern so re-running it updates existing rows instead of duplicating them. **api.py** is a FastAPI app that reads from the same database and exposes it over REST, deployed on Render. **dashboard.py** is a Streamlit app that also reads directly from the database and renders it as tables and charts, deployed on Streamlit Community Cloud. The API and dashboard use separate dependency files (requirements-api.txt vs. requirements.txt) since the API doesn't need Streamlit/Plotly at all.

Layer	Technology
Data source	yfinance (Yahoo Finance)
Database	SQLite
Backend API	FastAPI + Uvicorn
Frontend	Streamlit + Plotly
Hosting	Render (API) / Streamlit Community Cloud (dashboard)

APIs Used

GET /stocks — returns all 20 tracked companies with price, market cap, P/E ratio, and EPS. **GET /stocks/{ticker}** — returns a single company by ticker (e.g. /stocks/RELIANCE.NS), 404 if not tracked. **GET /market-summary** — returns aggregate stats (total stocks, average P/E, largest market cap) computed via SQL. Interactive docs auto-generated by FastAPI at /docs.

Database Design

A single SQLite table, **stocks**, with columns: ticker (primary key), company_name, price, market_cap, pe_ratio, eps, fetched_at. Using ticker as the primary key is what enables the upsert behavior — re-fetching data updates existing rows rather than growing the table indefinitely.

Features Implemented

MVP: live + stored data for 20 companies across sectors (banking, IT, FMCG, auto, pharma, cement); SQLite storage with upsert; 3 REST endpoints; dashboard with sortable overview table, historical price chart with adjustable time range, multi-company % change comparison, and fundamentals bar chart.
Beyond MVP: line/candlestick toggle for historical charts (using OHLC data yfinance already returns); a dashboard refresh button that re-runs the same fetch functions as algo1.py rather than duplicating logic; API and dashboard deployed as fully separate services.

Challenges Faced

Environment/deployment setup and the underlying concepts (REST APIs, SQL, databases) were about equally difficult, since this was a first end-to-end build. A specific issue: the initial requirements.txt was generated via pip freeze and pulled in the entire local environment, including packages the API doesn't use. This caused the Render build to fail while compiling pyarrow (an indirect Streamlit/Plotly dependency). Fixed by splitting into a minimal API-only requirements-api.txt and a separate dashboard requirements.txt — which also made more architectural sense, since the two services have no reason to share dependencies.

Future Improvements

Store historical price data in the database instead of querying yfinance live on every chart load (would also let the API serve historical data); scheduled automatic data refresh instead of a manual trigger; user authentication and personal watchlists; additional fundamentals beyond P/E and EPS (debt-to-equity, ROE).

AI Usage Disclosure: Claude (Anthropic) was used to explain unfamiliar concepts (virtual environments, REST APIs, SQL upserts, deployment), debug errors as they came up, and help structure this report and the project README. All code was written and tested by me, and I can explain the architectural decisions directly — the upsert logic, the dependency split, and the normalization approach used for the comparison chart.